

-IPFS-

۱. مقدمه

- در این پروژه یک سیستم ذخیره سازی فایل بر اساس محتوا پیاده سازی شده است که مشابه IPS است؛ منظور از "بر اساس محتوا بودن" این است که فایل ها بر اساس content خود ذخیره میشوند و نه بر اساس آدرس فیزیکی شان.

- این پروژه چه مشکلی را حل میکند؟

در این مدل پیاده سازی، هر فایل به تکه های ۲۵۶ کیلوبایتی به نام chunk تقسیم میشود. فرض کنید میخواهیم یک فایل ۱GB را آپلود کنیم. در روش بدون چانک کردن، باید کل فایل را هاش کنیم و سپس آن را سیو کنیم؛ واضح است که این روش بسیار کند است. همچنین برای دانلود آن، باید منتظر بمانیم تا کل فایل آپلود شود و سپس عملیات دانلود را شروع کنیم. برای حل این مشکل از چانک کردن استفاده میکنیم. فایل را به تعدادی چانک ۲۵۶KB تقسیم میکنیم؛ سپس هر چانک را به صورت موازی و مولتی ترد هاش میکنیم. اگر ۸ ترد برای انجام این کار داشته باشیم، این متد ۸ برابر سریع تر از روش قبلی است!

برتری دیگر این روش پیاده سازی به ویژگی content addressed بودن آن
برمیگردد. فرض کنید در روش Location Based، فایل در آدرس
"C:\Users\ASUS\Downloads\FUM-OS-1404-01-HW#1.pdf" قرار دارد.
اگر لووکیشن فایل تغییر کند، این آدرس دیگر معتبر نخواهد بود. اما در روش
content addressed، هر فایل با یک cid یونیک شناخته میشود. از مزیت های
این روش میتوان به این اشاره کرد که دو فایل با محتوای یکسان دقیقاً یک cid
دارند. همچنین این مدل دیگر به لووکیشن فایل وابسته نیست.

۲. معماری

- به طور کلی سیستم شامل دو فایل است: فایل [main.py](#) که نقش gateway
سیستم را دارد و فایل c_engine که هسته سیستم است. فایل پایتون شامل دو
اندپوینت upload و download است که توسط کلاینت روی پورت ۹۰۰۰ با پروتکل
HTTP کال میشوند؛ سپس gateway ریکوئست را از طریق unix socket به فایل
c که حاوی منطق اصلی سیستم است میفرستد و پس از پردازش-هش کردن در
در چند ترد، ذخیره کردن چانک ها، ساخت مانیفست و ... مجدداً از طریق unix
socket پاسخ را gateway ارسال میکند و gateway هم از طریق HTTP
ریسپانس را به کلاینت برمیگرداند.
- علاوه بر این دو فایل، تعدادی اسکریپت پایتون برای تست کردن سیستم در
سناریوهای مختلف و یک فایل [gui.py](#) هم داریم که پیاده سازی رابط کاربری
گرافیکی در آن با استفاده از کتابخانه stream lit انجام شده است.

۳. مراحل پیاده سازی

۳-۱. مرحله اول به نوشتن توابع زیرساختی پروژه مثل ساخت دایرکتوری، محاسبه هاش، encoding، محاسبه cid، توابع و ساختار مربوط به بلاک ها و ... گذشت که در ادامه به جزئیات پیاده سازی هر بخش میپردازیم.

- هاش کردن:

برای عملیات هاش کردن، علی رغم این که در صورت پروژه پیشنهاد شده بود که از الگوریتم BLAKE-۳ استفاده کنیم، ما از الگوریتم SHA-۲۵۶ استفاده کردیم. یکی از مهم ترین دلایل این انتخاب سادگی و دسترسی بیشتر این الگوریتم نسبت به الگوریتم BLAKE-۳ است. در مقیاسی که ما استفاده میکنیم، این انتخاب تاثیر خاصی روی اهداف یادگیری پروژه مثل concurrency، مدیریت تردها و فرایندها، و ارتباط بین فرایندها (IPC) ندارد.

مکانیزم هاش کردن در تابع compute_multihash نوشته شده که هر چانک را با استفاده از الگوریتم SHA-۲۵۶ هاش میکند و یک prefix به آن اضافه میکند که مشخص کننده الگوریتم هاش کردن است؛ در اینجا این پیشوند 0x1220 است.

```
// Compute multihash: prefix (0x1220 for sha256) + hash
char* compute_multihash(const uint8_t* data, size_t len) {
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256(data, len, hash);

    // Multihash format: 0x12 (sha256) + 0x20 (32 bytes) + hash
    char* mhash = malloc(sizeof(char) * (4 + SHA256_DIGEST_LENGTH * 2 + 1));
    if (!mhash) return NULL;

    sprintf(mhash, "1220");
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++) {
        sprintf(mhash + 4 + (i * 2), "%02x", hash[i]);
    }
    mhash[4 + SHA256_DIGEST_LENGTH * 2] = '\0';

    return mhash;
}
```

- تابع `base32_encoding` هم دیتای باینری را به کاراکترهای خوانا تبدیل میکند؛ از این تابع در محاسبه `cid` استفاده میشود.

```
235 // Simple base32 encoding (lowercase)
236 static const char base32_chars[] = "abcdefghijklmnopqrstuvwxyz234567";
237
238 char* base32_encode(const uint8_t* data, size_t len) {
239     size_t output_len = ((len * 8 + 4) / 5);
240     char* output = malloc(size:output_len + 1);
241     if (!output) return NULL;
242
243     size_t bit_buffer = 0;
244     int bits_in_buffer = 0;
245     size_t output_pos = 0;
246
247     for (size_t i = 0; i < len; i++) {
248         bit_buffer = (bit_buffer << 8) | data[i];
249         bits_in_buffer += 8;
250
251         while (bits_in_buffer >= 5) {
252             output[output_pos++] = base32_chars[(bit_buffer >> (bits_in_buffer - 5)) & 0x1F];
253             bits_in_buffer -= 5;
254         }
255     }
256
257     if (bits_in_buffer > 0) {
258         output[output_pos++] = base32_chars[(bit_buffer << (5 - bits_in_buffer)) & 0x1F];
259     }
260
261     output[output_pos] = '\0';
262     return output;
263 }
264
```

دلیل این که از کاراکترهای 0 و 1 و اعداد ۰ و ۱ استفاده نشده، شباهت های آنها به یکدیگر است.

- پس از هاش شدن چانک ها، نوبت به ساخت cid میرسد. همان طور که در داک پروژه اشاره شده، cid از سریالایز کردن manifest، سپس هاش کردن آن و در نهایت encode کردن خروجی به دست می آید.

```
// Compute CID from manifest JSON
char* compute_cid(const char* manifest_json) {
    // Hash the manifest
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)manifest_json, strlen(manifest_json), hash);

    // Encode in base32
    return base32_encode(hash, sizeof hash);
}
```

در تابع compute_cid ابتدا فایل manifest خوانده می شود و سپس از آن هاش گرفته میشود و در نهایت نتیجه آن به تابع base32-encode میرود. نتیجه نهایی یک رشته یونیک است که cid نام دارد. این cid در دایرکتوری manifests ذخیره میشود و در عملیات دانلود از روی همین مقدار کل فایل بازسازی میشود.

- برای سیو کردن چانک ها از یک ساختار content-addressed استفاده شده؛ یعنی هر چانک فقط بر اساس هش خودش ذخیره میشود و نه نام فایل اصلی.

```
// Get block file path from multihash
// Example: "1220abcd..." -> "blocks/ab/cd/1220abcd..."
char* get_block_path(const char* multihash) {
    if (strlen(multihash) < 8) return NULL;

    char* path = malloc(sizeof(char)*512);
    if (!path) return NULL;

    // Extract first 4 hex chars (after "1220" prefix)
    char dir1[3] = {multihash[4], multihash[5], '\0'};
    char dir2[3] = {multihash[6], multihash[7], '\0'};

    snprintf(path, sizeof(path), "blocks/%s/%s/%s", dir1, dir2, multihash);
    return path;
}

// Save block to disk
int save_block(const char* multihash, const uint8_t* data, size_t len) {
    char* path = get_block_path(multihash);
    if (!path) return -1;

    // Create parent directories
    char dir[512];
    snprintf(dir, sizeof(dir), "%s", path);
    char* last_slash = strrchr(dir, '/');
    if (last_slash) {
        *last_slash = '\0';
        if (mkdirs(dir) != 0) {
            free(path);
            return -1;
        }
    }
}
```

در تابع save_block ابتدا هش چانک بررسی میشود و یک مسیر مانند زیر

ساخته میشود: "blocks/ab/cd/hash"

دو بخش ab و cd برای جلوگیری از شلوغی دایرکتوری اصلی استفاده میشوند. پس از ساخت فولدر، اگر فایل چانک موجود نباشد، ایجاد میشود؛ و اگر وجود داشته باشد، نیاز به ذخیره دوباره نیست، چون همین محتوا قبلا آپلود شده است. این نوع پیاده سازی به صرفه جویی در حافظه کمک میکند.

۲-۳. در این مرحله اجزای زیرساختی برای پشتیبانی از عملیات موازی و مدیریت نشست ها نوشته شدند.

- ساختار داده‌ها: برای مدیریت بلاک‌ها، manifest، تردها و نشست ها چند ساختار تعریف شده است.

```
// Work item for thread pool
typedef struct {
    work_type_t type;
    void* session;          // upload_session_t* or download_session_t*
    uint32_t index;
    uint8_t* data;
    size_t size;
    char expected_hash[MAX_HASH_LEN];
} work_item_t;

// Thread pool
typedef struct {
    pthread_t* threads;
    size_t num_threads;

    work_item_t* queue;
    size_t queue_head;
    size_t queue_tail;
    size_t queue_size;
    size_t queue_capacity;

    pthread_mutex_t mutex;
    pthread_cond_t cond;
    pthread_cond_t empty_cond;

    int shutdown;
    int active_tasks;
} thread_pool_t;
```

- مدیریت جدول بلاک‌ها: برای جلوگیری از ذخیره چندباره ی یک بلاک (در شرایطی که محتوا تکراری باشد)، یک جدول برای نگهداری بلاک‌ها ایجاد شد. بلوک‌ها با هاش شناسایی میشوند؛ اگر یک هاش قبلاً ثبت شده باشد، تنها refcount آن افزایش پیدا می‌کند؛ با این پیاده سازی وقتی فایل های مشابه آپلود میکنیم، عملیات اضافه انجام نمی‌شود و I/O هارد کاهش پیدا میکند. این جدول با یک قفل محافظت می‌شود تا چندین ترد نتوانند همزمان روی آن تغییر اعمال کنند.

```
// Global block table
typedef struct {
    block_entry_t* entries;
    size_t count;
    size_t capacity;
    pthread_rwlock_t lock;
} block_table_t;
```

```
// Add or increment refcount
void block_table_add_or_inc(const char* hash) {
    pthread_rwlock_wrlock(&g_block_table.lock);

    int idx = block_table_find(hash);
    if (idx >= 0) {
        // Block exists, increment refcount
        g_block_table.entries[idx].refcount++;
    } else {
        // New block
        if (g_block_table.count >= g_block_table.capacity) {
            g_block_table.capacity *= 2;
            g_block_table.entries = realloc(g_block_table.entries,
                                             size_t:g_block_table.capacity * sizeof(block_entry_t));
        }
        strncpy(g_block_table.entries[g_block_table.count].hash, hash, MAX_HASH_LEN - 1);
        g_block_table.entries[g_block_table.count].refcount = 1;
        g_block_table.count++;
    }

    pthread_rwlock_unlock(&g_block_table.lock);
}

// Check if block exists
int block_table_exists(const char* hash) {
    pthread_rwlock_rdlock(&g_block_table.lock);
    int idx = block_table_find(hash);
    pthread_rwlock_unlock(&g_block_table.lock);
    return idx >= 0;
}
```

- در این مرحله همچنین به پیاده سازی Thread Pool پرداختیم. دلیل استفاده از Thread Pool هم این است که نمیخواهم هر بار که میخواهیم یک چانک را پردازش کنیم یک ترد بسازیم؛ این کار اوردهد میتواند داشته باشد. بنابراین در ابتدا ۸ ترد داریم که همواره منتظر یک کارند، وقتی یک کار جدید به صف اضافه میشود، condition variable تردها را بیدار میکند.

- هر عملیات آپلود یک نشست جدید ایجاد میکند. در این نشست اطلاعات زیر نگهداری می‌شود:

```
// Upload session (per connection)
typedef struct {
    char filename[MAX_FILENAME];
    uint64_t total_size;
    uint32_t chunk_size;
    chunk_info_t* chunks;
    size_t chunk_count;
    size_t chunk_capacity;
    pthread_mutex_t lock;
} upload_session_t;
```

سشن تا زمانی فعال است که پیام UPLOAD_FINISH برسد و همه‌ی چانک‌ها پردازش شده باشند. پس از آن manifest نوشته شده و cid محاسبه می‌شود.

- و در نهایت در انتهای این مرحله باید با توجه به تغییراتی که انجام دادیم، تابع main را آپدیت کنیم. در تابع main، قبل از شروع به accept کردن کانکشن‌ها، دایرکتوری‌های blocks و manifests ایجاد می‌شود، جدول بلاک‌ها initialize می‌شود، Thread pool ساخته می‌شود، ساختارهای مدیریت session آماده می‌شوند و در نهایت سرور روی tmp/engine.sock/ باید می‌شود. به این ترتیب engine اصلی سیستم قبل از پذیرش اولین درخواست آماده می‌شود.

```

5 int main(int argc, char** argv) {
6     if (argc != 2) {
7         fprintf(stderr, "usage: %s /tmp/engine.sock\n", argv[0]);
8         return 2;
9     }
10    g_sock_path = argv[1];
11
12    // Create directories
13    mkdirs(path: "blocks");
14    mkdirs(path: "manifests");
15
16    // Initialize global structures
17    block_table_init();
18    g_thread_pool = thread_pool_create(num_threads: THREAD_POOL_SIZE);
19
20    printf(format: "[ENGINE] Thread pool initialized with %d threads\n", THREAD_POOL_SIZE);
21
22    int fd = socket(domain: AF_UNIX, type: SOCK_STREAM, protocol: 0);
23    if (fd < 0) { perror(s: "socket"); return 2; }
24
25    struct sockaddr_un addr;
26    memset(&addr, 0, sizeof(addr));
27    addr.sun_family = AF_UNIX;
28    strncpy(addr.sun_path, g_sock_path, sizeof(addr.sun_path) - 1);
29    unlink(g_sock_path);
30
31    if (bind(fd, (struct sockaddr*)&addr, len: sizeof(addr)) < 0) { perror(s: "bind"); return 2; }
32    if (listen(fd, 64) < 0) { perror(s: "listen"); return 2; }

```

- ۳-۳. در مرحله سوم، فرایند آپلود را در تابع handle-connection پیاده سازی میکنیم.
- نقطه ابتدایی فرایند آپلود فرستادن ریکوئست HTTP از طرف کلاینت به gateway است. در ادامه داکيومنت راجع به طریقه صدا زدن اندپوینت های آپلود و دانلود صحبت خواهیم کرد.
 - وقتی opcode برابر UPLOAD_START باشد، یک upload session جدید ساخته میشود:

```

if (op == OP_UPLOAD_START) {
    // Extract filename from payload
    char filename[MAX_FILENAME];
    size_t fn_len = (len < MAX_FILENAME - 1) ? len : MAX_FILENAME - 1;
    memcpy(filename, payload, fn_len);
    filename[fn_len] = '\0';

    printf(format: "[ENGINE] UPLOAD_START: name=\"%s\"\n", filename);
    fflush(stdout);

    // Create upload session
    upload_session = upload_session_create(filename);
    if (!upload_session) {
        send_error(fd: cfd, code: E_BUSY, message: "Cannot create upload session");
        free(payload);
        break;
    }
}

```

- سپس در پیام های بعدی اگر opcode برابر با **UPLOAD_CHUNK** بود، بایت های چانک خوانده میشوند(در این زمان قبل از هر تغییری در session، قفل را فعال میکنیم تا race condition پیش نیاید؛ بدون این مکانیزم، دو چانک میتوانند ایندکس های یکسانی بگیرند. بعد از این که عملیات روی نشست تمام شد و هر چانک ایندکس یونیک خودش را گرفت، قفل را باز میکنیم)، یک job برای thread pool ساخته میشود و به thread pool سابمیت میشود تا یک worker thread آن را بردارد(برای این کار هم ابتدا قفل را میبندیم تا دو کار به یک ترد نرسد). سپس تردها به صورت موازی هر چانک را پردازش میکنند. هر worker thread ابتدا چانک را هاش میکند، جدول بلاک ها را چک میکند که ببیند اگر موجود است آن را مجددا ذخیره نکند و صرفا متغیر refcount را یکی افزایش دهد؛ سپس مجددا سشن را قفل میکند تا هاش چانک را در ساختار همان session بنویسد؛ سپس قفل را باز میکند و چانک را آزاد میکند.

```
for (;;) {
    } else if (op == OP_UPLOAD_CHUNK) {
        // LOCK before accessing/modifying session
        pthread_mutex_lock(&upload_session->lock);

        // Expand chunk array if needed
        if (upload_session->chunk_count >= upload_session->chunk_capacity) {
            upload_session->chunk_capacity *= 2;
            chunk_info_t* new_chunks = realloc(upload_session->chunks,
                sizeof(chunk_info_t) * upload_session->chunk_capacity);
            if (!new_chunks) {
                pthread_mutex_unlock(&upload_session->lock);
                send_error(fd, cfd, code: E_BUSY, message: "Memory allocation failed");
                free(payload);
                break;
            }
            upload_session->chunks = new_chunks;
        }

        // Get chunk index (current count)
        uint32_t chunk_idx = upload_session->chunk_count;

        // PRE-INCREMENT chunk_count (IMPORTANT!)
        upload_session->chunk_count++;

        // Update total size
        upload_session->total_size += len;

        pthread_mutex_unlock(&upload_session->lock);
    }
}
```

```

// Create work item (outside lock)
work_item_t work;
work.type = WORK_HASH_AND_SAVE;
work.session = upload_session;
work.index = chunk_idx;
work.data = payload; // Thread will free this
work.size = len;
work.expected_hash[0] = '\0';

// Submit to thread pool
thread_pool_submit(g_thread_pool, work);

// Don't free payload here - worker thread will free it
payload = NULL;

```

تابع worker thread که راجع به آن صحبت کردیم:

```

1 void* worker_thread(void* arg) {
2     thread_pool_t* pool = (thread_pool_t*)arg;
3
4     while (1) {
5         pthread_mutex_lock(&pool->mutex);
6
7         // Wait for work
8         while (pool->queue_size == 0 && !pool->shutdown) {
9             pthread_cond_wait(&pool->cond, &pool->mutex);
10        }
11
12        if (pool->shutdown && pool->queue_size == 0) {
13            pthread_mutex_unlock(&pool->mutex);
14            break;
15        }
16
17        // Get work item
18        work_item_t work = pool->queue[pool->queue_head];
19        pool->queue_head = (pool->queue_head + 1) % pool->queue_capacity;
20        pool->queue_size--;
21        pool->active_tasks++;
22
23        pthread_mutex_unlock(&pool->mutex);
24
25        // Process work
26        if (work.type == WORK_HASH_AND_SAVE) {
27            // Hash the chunk
28            char* mhash = compute_multihash(work.data, work.size);
29
30            // Check if block already exists
31            if (!block_table_exists(mhash)) {
32                // Save new block
33                save_block(mhash, work.data, work.size);
34            }
35
36            // Add to block table (or increment refcount)
37            block_table_add_or_inc(mhash);
38
39            // Update session
40            upload_session_t* session = (upload_session_t*)work.session;
41            pthread_mutex_lock(&session->lock);
42
43            if (work.index < session->chunk_capacity) {
44                session->chunks[work.index].index = work.index;
45                session->chunks[work.index].size = work.size;
46                strncpy(session->chunks[work.index].hash, mhash, MAX_HASH_LEN - 1);
47                // DON'T update chunk_count here - it's already set in the main thread
48            }
49
50            pthread_mutex_unlock(&session->lock);
51
52            free(mhash);
53            free(work.data);
54        }
55    }
56}

```

- و در نهایت اگر opcode برابر با **UPLOAD_FINISH** بود، وقتی همه تردها کارشان را تمام کردند و همه چانک ها رسیدند، مانیفست ساخته میشود، cid محاسبه میشود، تابع `save_monifest` صدا زده میشود و مانیفست ذخیره میشود(مرحله renaming آن از `json.temp.` به `json.` به صورت اتمیک انجام میشود که عکس آن در ادامه آورده شده است) و در نهایت پیام **UPLOAD_DONE** همراه با cid برای gateway ارسال میشود تا cid به دست کلاینت برسد. به این ترتیب آپلود کامل میشود.

تابع `save_monifest`:

```
int save_manifest(const char* cid, const char* json) {
    char tmp_path[512];
    char final_path[512];

    snprintf(tmp_path, sizeof(tmp_path), "manifests/%s.json.tmp", cid);
    snprintf(final_path, sizeof(final_path), "manifests/%s.json", cid);

    // Write to temp file
    FILE* f = fopen(tmp_path, "w");
    if (!f) return -1;

    fwrite(json, sizeof(char), strlen(json), f);
    fflush(f);
    fsync(fileno(f));
    fclose(f);

    // Atomic rename
    if (rename(tmp_path, final_path) != 0) {
        unlink(tmp_path);
        return -1;
    }

    return 0;
}
```

۳-۴. و در نهایت در مرحله چهارم، فرایند دانلود را پیاده سازی کردیم. ساختار دانلود در مقایسه با آپلود ساده تر است، اما همچنان نیاز به مدیریت session دارد. نقطه ابتدایی فرایند دانلود، کال کردن اندپوینت دانلود توسط کلاینت و صدا زدن آن با متد GET است که در ادامه راجع به آن صحبت خواهیم کرد.

- در این فرایند، ابتدا cid فایل مورد نظر extract میشود؛ پس از validation آن، با تابع download_session_create یک session جدید ساخته می‌شود. این تابع ابتدا فایل مانیفست مورد نظر را می‌خواند، json را پارس میکند، لیست و تعداد چانک ها را extract میکند و در نهایت بافرهایی را به چانک ها اختصاص میدهد. بنا بر این پس از این اتفاقات، download_session شامل cid، تعداد چانک ها و اطلاعات مربوط به هر چانک، بافرها، ایندکسی برای چانک بعدی و آرایه ready است که با تمام شدن کار هر ترد، خانه شماره آن چانک را یک میکند؛ از این آرایه برای منطق به ترتیب فرستادن چانک ها استفاده میشود.

```
} else if (op == OP_DOWNLOAD_START) {
    char cid[256];
    size_t cid_len = (len < 255) ? len : 255;
    memcpy(cid, payload, cid_len);
    cid[cid_len] = '\0';

    printf(format: "[ENGINE] DOWNLOAD_START: cid=\"%s\"\n", cid);
    fflush(stdout);

    // Validate CID format
    if (!is_valid_cid(cid)) {
        printf(format: "[ENGINE] ERROR: Invalid CID format\n");
        fflush(stdout);
        send_error(fd: cfd, code: E_BAD_CID, message: "Invalid CID format");
        free(payload);
        continue;
    }

    // Create download session
    download_session_t* download_session = download_session_create(cid);
    if (!download_session) {
        send_error(fd: cfd, code: E_NOT_FOUND, message: "CID not found or invalid manifest");
        free(payload);
        continue;
    }

    printf(format: "[ENGINE] Found %zu chunks, submitting for verification...\n",
           download_session->total_chunks);
```

پس از آن چانک ها را به thread pool سابمیت میکند تا توسط worker thread ها به صورت موازی verify شوند.

```
// Submit all chunks for parallel verification
for (size_t i = 0; i < download_session->total_chunks; i++) {
    work_item_t work;
    work.type = WORK_LOAD_AND_VERIFY;
    work.session = download_session;
    work.index = i;
    work.data = NULL;
    work.size = 0;
    strncpy(work.expected_hash, download_session->chunks[i].hash, (MAX_HASH_LEN - 1));
    work.expected_hash[MAX_HASH_LEN - 1] = '\0';

    thread_pool_submit(g_thread_pool, work);
}
```

- هر worker thread ابتدا بلاک مربوطه را از دیسک لود میکند، هاش آن را verify میکند، سپس بافرها را با دیتای مربوطه پر میکند و ready آن را یک میکنیم. برای این عملیات هم از مکانیزم synchronization استفاده میکنیم تا از نوشتن دیتای اشتباه جلوگیری کنیم.

```
} else if (work.type == WORK_LOAD_AND_VERIFY) {
    if (!block_data) {
        // Block not found - mark as error
        download_session_t* session = (download_session_t*)work.session;
        pthread_mutex_lock(&session->lock);
        session->ready[work.index] = -1; // Error marker
        pthread_cond_broadcast(&session->cond);
        pthread_mutex_unlock(&session->lock);
    } else {
        // Verify hash
        char* computed_hash = compute_multihash(block_data, block_size);
        int hash_match = (strcmp(computed_hash, work.expected_hash) == 0);
        free(computed_hash);

        download_session_t* session = (download_session_t*)work.session;
        pthread_mutex_lock(&session->lock);

        if (hash_match) {
            // Store verified chunk
            session->buffers[work.index] = block_data;
            session->sizes[work.index] = block_size;
            session->ready[work.index] = 1;
        } else {
            // Hash mismatch
            free(block_data);
            session->ready[work.index] = -1;
        }

        pthread_cond_broadcast(&session->cond);
        pthread_mutex_unlock(&session->lock);
    }
}
```

- در این بخش راجع به به ترتیب ارسال کردن چانک ها صحبت میکنیم. برای این کار، مثلاً برای چانک 0 ام تا زمانی که مقدار خانه صفرم آرایه ready یک نشده است منتظر میمانیم. همین مکانیزم را به ترتیب برای همه چانک ها اجرا میکنیم.

```
// Stream chunks in order
for (size_t i = 0; i < download_session->total_chunks; i++) {
    pthread_mutex_lock(&download_session->lock);

    // Wait for chunk to be ready
    while (download_session->ready[i] == 0) {
        pthread_cond_wait(&download_session->cond, &download_session->lock);
    }

    if (download_session->ready[i] < 0) {
        // Error occurred
        pthread_mutex_unlock(&download_session->lock);
        send_error(cf, code: E_HASH_MISMATCH, message: "Chunk verification failed");
        download_session_destroy(download_session);
        free(payload);
        goto connection_end;
    }

    // Send chunk
    send_frame(cf, op: OP_DOWNLOAD_CHUNK,
               download_session->buffers[i],
               download_session->sizes[i]);

    pthread_mutex_unlock(&download_session->lock);
}
```

و در نهایت OP_DOWNLOAD_DONE ارسال میشود.

۴. در این قسمت راجع به نحوه اجرای برنامه صحبت میکنیم:

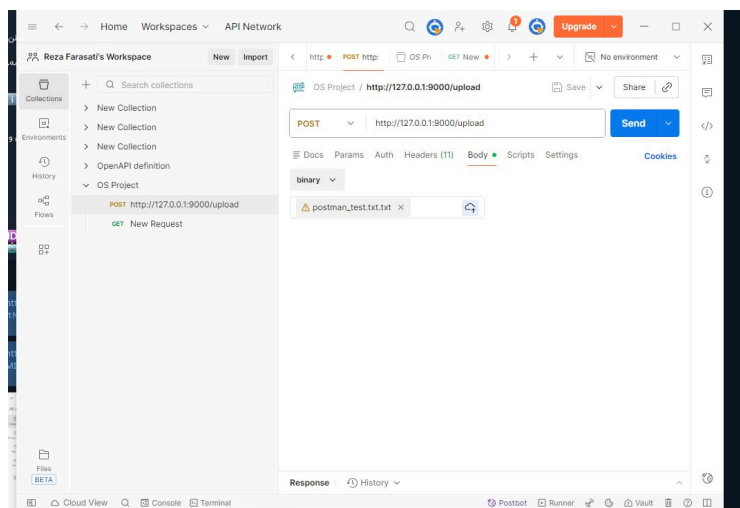
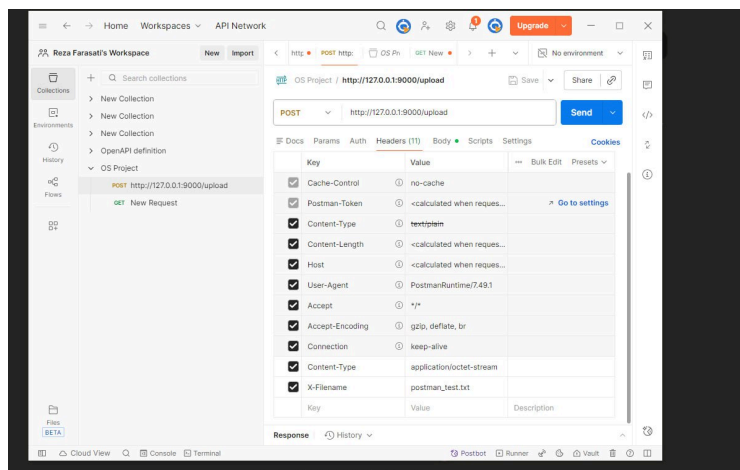
- ابتدا در یک ترمینال کامند زیر را ران میکنیم تا یک unix socket بسازد:

```
reza@Almas-PC:/mnt/c/Users/ASUS/CLionProjects/os_midterm_project$ ./c_engine /tmp/engine.sock  
[ENGINE] Thread pool initialized with 8 threads  
[ENGINE] listening on /tmp/engine.sock
```

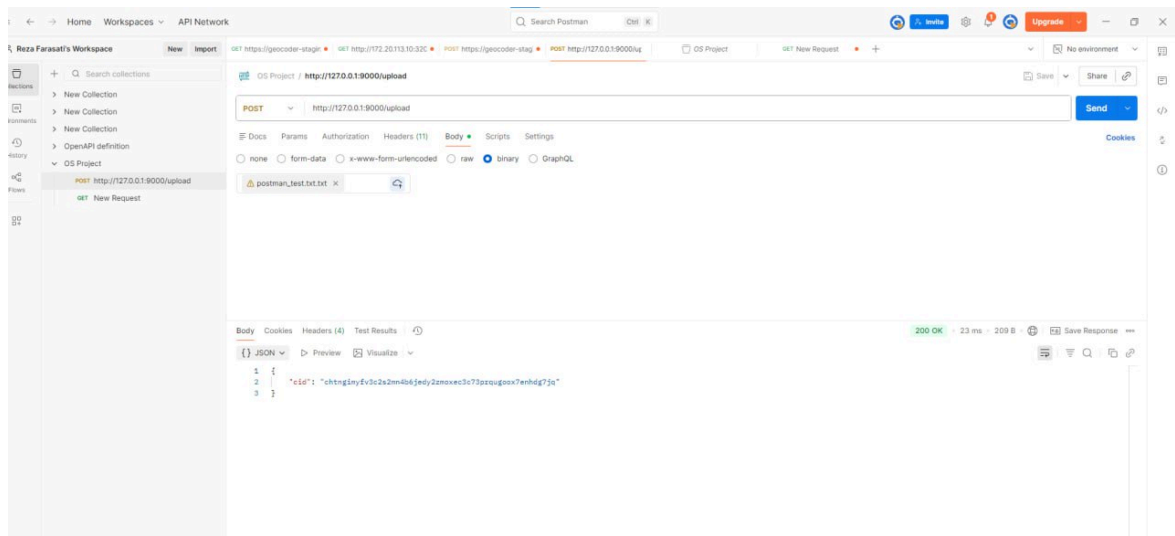
- در ترمینال دیگر gateway را ران میکنیم:

```
^Creza@Almas-PC:/mnt/c/Users/ASUS/CLionProjects/os_midterm_project$ python3 main.py  
HTTP gateway listening on http://127.0.0.1:9000
```

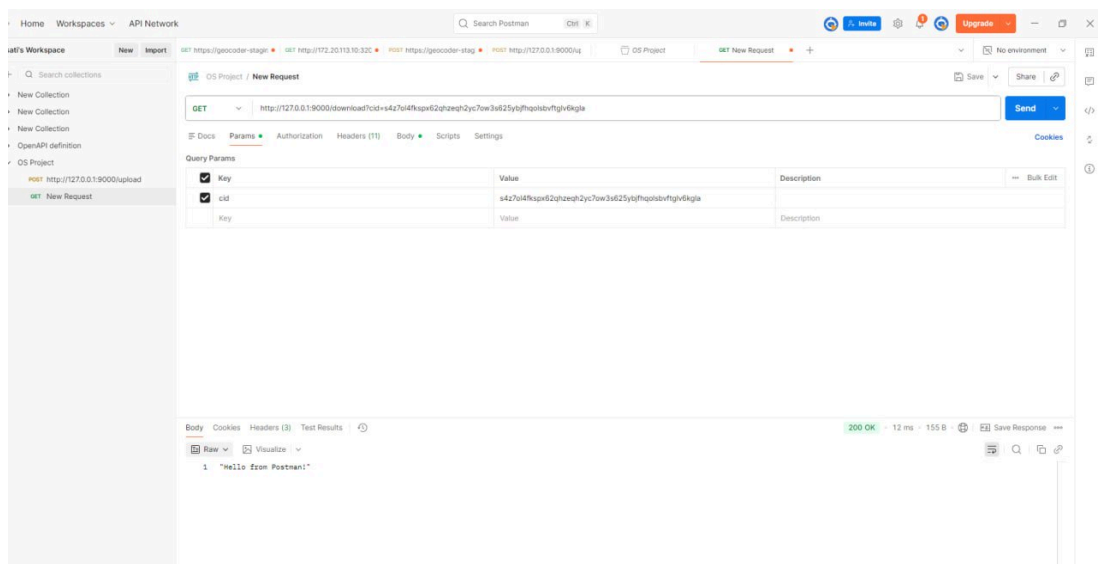
- حال بسته به تستی که میخواهیم انجام دهیم کارمان متفاوت است. برای یک ریکوئست ساده، ابتدا یک فایل text میسازیم و از طریق اپلیکیشن postman با هدرهای مربوطه فایل را آپلود میکنیم و اندپوینت آپلود را صدا میزنیم:



در پاسخ cid برگردانده میشود:



- برای دانلود هم باید cid را به عنوان param ست کنیم و این بار با متد GET به اندپوینت دانلود ریکوئست بزنیم:



در پاسخ محتوای فایل برگردانده میشود.

- برای تست سناریوهای مختلف، اسکریپت های پایتون ران شدند که در ادامه توضیحات لازم ارائه خواهد شد:

- تست ۱: Concurrent Uploads

در این تست چند فایل متفاوت به صورت همزمان آپلود میشوند. این سناریو صحت Thread Pool، قفل ها و مدیریت نشست را بررسی میکند. تمام آپلودها با Status 200 و cid یونیک درست تمام شدند.

- تست ۲: Concurrent Downloads

در این مرحله یک فایل آپلود شده و سپس توسط چندین thread همزمان دانلود شد. این تست صحت Reader Lock و مدیریت همزمان خواندن بلاک ها را میسنجد. تمام دانلودها موفق بودند و محتوا بدون اختلاف بازسازی شد.

- تست ۳: Deduplication

در این تست دو فایل کاملاً یکسان اما با نام های متفاوت آپلود شدند. انتظار این است که بلاک های مشترک دوباره ذخیره نشوند. نتیجه مطابق انتظار بود: تعداد بلاک ها بعد از آپلود دوم ثابت ماند و deduplication صحیح کار کرد.

- تست ۴: Large File

یک فایل ۲ مگابایتی آپلود و سپس دانلود شد. هش نسخه اصلی و نسخه دانلود شده مقایسه شد و یکسان بودند. این تست اعتبار کل فرایندهای دخیل را برای فایل های بزرگ هم تایید میکند.

- تست ۵: Race Condition

در این تست سعی شد که لود زیادی روی سیستم گذاشته شود و به صورت همزمان ۲۰ فایل چندتکه ای را آپلود کردیم. این بخش برای ارزیابی استحکام mutex ها، Work queue و thread pool اهمیت دارد. تمام ۲۰ آپلود بدون crash، deadlock یا داده خراب به پایان رسیدند.

- تست ۶: Error Handling

چند سناریوی خطا مثل cid نامعتبر، cid ناموجود، دانلود بدون پارامتر cid، یا آپلود بدون اسم فایل اجرا شدند و نتایج قابل انتظار گرفته شد.

- همه تست ها به صورت centralized در فایل final_test_suit.py ران شدند و کل تست ها بدون خطا پاس شدند.

۵. و در نهایت در مرحله آخر رابط کاربری گرافیکی برای انجام عملیات آپلود و دانلود در فایل [gui.py](#) با استفاده از کتابخانه streamlit پیاده سازی شد.

ابتدا:

```
PS C:\Users\ASUS\CLionProjects\os_midterm_project\storage_gui> streamlit run gui.py
```

- عملیات آپلود:

System Status

Status: Connected

Debug Info

Connected to: WSL: Ubuntu (127.0.0.1:9000)

Upload Download About

Upload Files

Upload files to the content-addressed storage system. Files are automatically chunked and deduplicated.

Choose files to upload

Drag and drop files here
Limit 200MB per file

Browse files

test.txt 10.0B

×

Upload All

All uploads complete!

test.txt

qwd3bhapfwjypje7wvjgusa4lyvhuo1xgk4bcpxcvxtqxheaaa

Size: 10 bytes

Time: 0.11s

- عملیات دانلود:

System Status

Status: Connected

Debug Info

Download Files

Enter a CID to download the corresponding file.

Content Identifier (CID)

qwd3bhapfwjypje7wvjgusa4lyvhuo1xgk4bcpxcvxtqxheaaa

Download

File retrieved successfully!

Save as downloaded_qwd3bhapfwjypje7.bin

File Size

10 bytes

Chunks

1

File Preview (first 256 bytes)

48 69 69 69 69 69 69 69

Text Preview:

HHHHHH